Chapter 12 Exception Handling and Text IO



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Motivations

When a program runs into a runtime error, the program terminates abnormally. How can you handle the runtime error so that the program can continue to run or terminate gracefully? This is the subject we will introduce in this chapter.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Objectives

- To get an overview of exceptions and exception handling (§12.2).
- To explore the advantages of using exception handling (§12.2).
- To distinguish exception types: **Error** (fatal) vs. **Exception** (nonfatal) and checked vs. unchecked (§12.3).
- To declare exceptions in a method header (§12.4.1).
- To throw exceptions in a method (§12.4.2).
- To write a **try-catch** block to handle exceptions (§12.4.3).
- To explain how an exception is propagated (§12.4.3).
- To obtain information from an exception object (§12.4.4).
- To develop applications with exception handling (§12.4.5).
- To use the **finally** clause in a **try-catch** block (§12.5).
- To use exceptions only for unexpected errors (§12.6).
- To rethrow exceptions in a **catch** block (§12.7).
- To create chained exceptions (§12.8).
- To define custom exception classes (§12.9).



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Exception-Handling Overview

- Exceptions are runtime errors. Exception handling enables a program to deal with runtime errors and continue its normal execution.
- An *exception* is an object that represents an error or a condition that prevents execution from proceeding normally.
- *Exceptions are thrown from a method. The caller of the method can catch and handle the exception.*



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Exception-Handling Overview

Show runtime error

Quotient

Fix it using an if statement

QuotientWithIf

With a method

QuotientWithMethod



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Exception Advantages

QuotientWithException



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Exception-Handling

```
1  import java.util.Scanner;
2
3  public class QuotientWithException {
4    public static int quotient(int number1, int number2) {
5      if (number2 == 0)
6        throw new ArithmeticException("Divisor cannot be zero");
7
8      return number1 / number2;
9    }
10
11   public static void main(String[] args) {
12     Scanner input = new Scanner(System.in);
13
14     // Prompt the user to enter two integers
15     System.out.print("Enter two integers: ");
16     int number1 = input.nextInt();
17     int number2 = input.nextInt();
18
19     try {
20       int result = quotient(number1, number2);
21       System.out.println(number1 + " / " + number2 + " is "
22         + result);
23     }
24     catch (ArithmeticException ex) {
25       System.out.println("Exception: an integer " +
26         "cannot be divided by zero ");
27     }
28
29     System.out.println("Execution continues ...");
30   }
31 }
```

If an ArithmeticException occurs

The value thrown is called an *exception*.
The execution of a **throw** statement is called *throwing an exception.*

When an exception is thrown, the normal execution flow is interrupted.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Exception-Handling

```
1  import java.util.Scanner;
2
3  public class QuotientWithException {
4    public static int quotient(int number1, int number2) {
5      if (number2 == 0)
6        throw new ArithmeticException("Divisor cannot be zero");
7
8      return number1 / number2;
9    }
10
11   public static void main(String[] args) {
12     Scanner input = new Scanner(System.in);
13
14     // Prompt the user to enter two integers
15     System.out.print("Enter two integers: ");
16     int number1 = input.nextInt();
17     int number2 = input.nextInt();
18
19     try {
20       int result = quotient(number1, number2);
21       System.out.println(number1 + " / " + number2 + " is "
22         + result);
23     }
24     catch (ArithmeticException ex) {
25       System.out.println("Exception: an integer " +
26         "cannot be divided by zero ");
27     }
28
29     System.out.println("Execution continues ...");
30   }
31 }
```

If an ArithmeticException occurs

The **try** block contains the code that is executed in normal circumstances.

The exception is caught by the **catch** block.

The code in the **catch** block is executed to *handle the exception*. Afterward, the statement after the **catch** block is executed.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

A template for a **try-throw-catch** block

```
try {
    Code to run;
    A statement or a method that may throw
    an exception;
    More code to run;
}
catch (type ex) {
     Code to process the exception;
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Exception Advantages

- Now you see the *advantages* of using exception handling.
- It enables a method to throw an exception to its caller.
- Without this capability, a method must handle the exception or terminate the program.
- An exception may be thrown directly by using a **throw** statement in a **try** block, or by invoking a method that may throw an exception.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Handling InputMismatchException

InputMismatchExceptionDemo

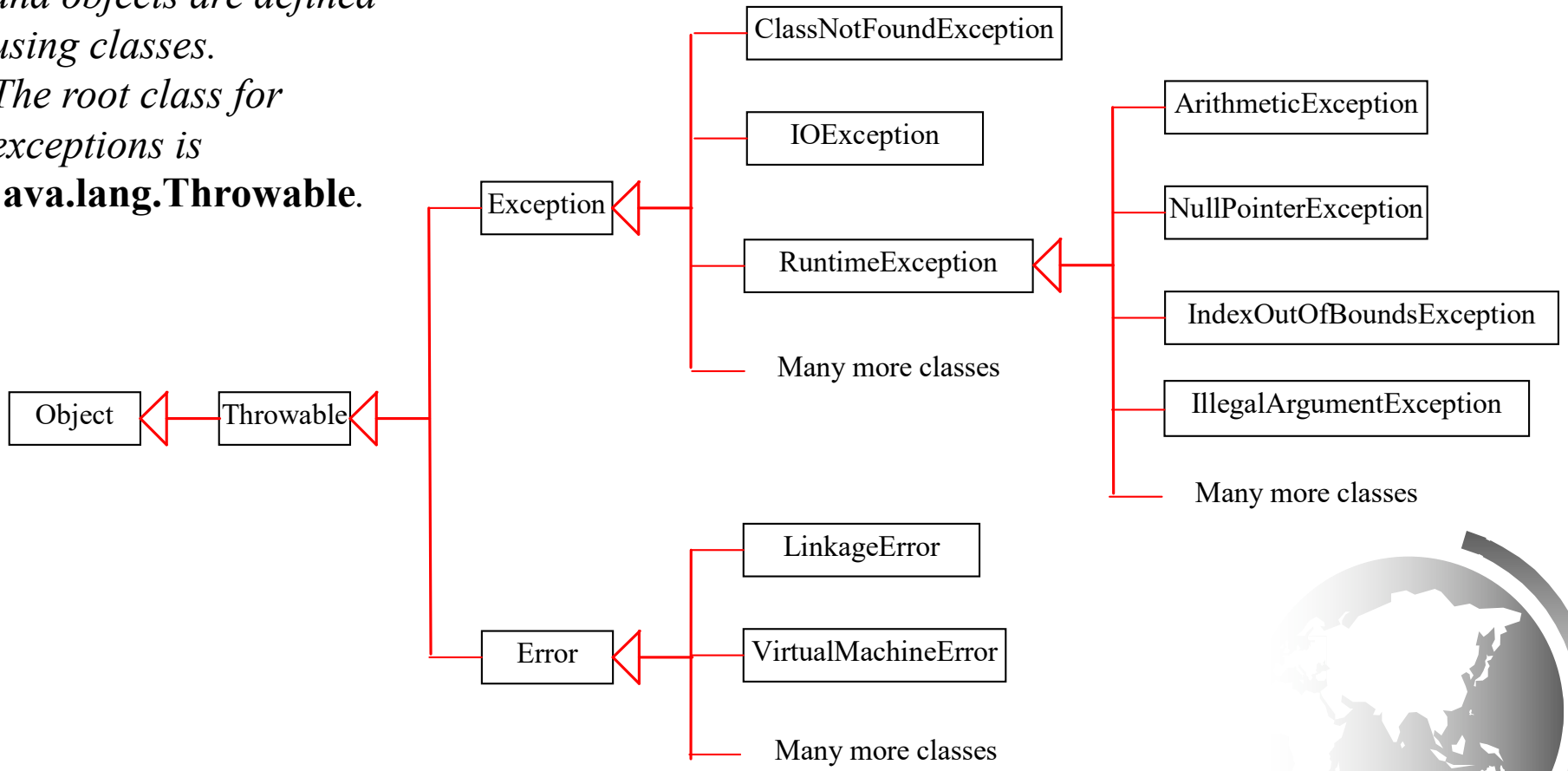
By handling InputMismatchException, your program will continuously read an input until it is correct.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

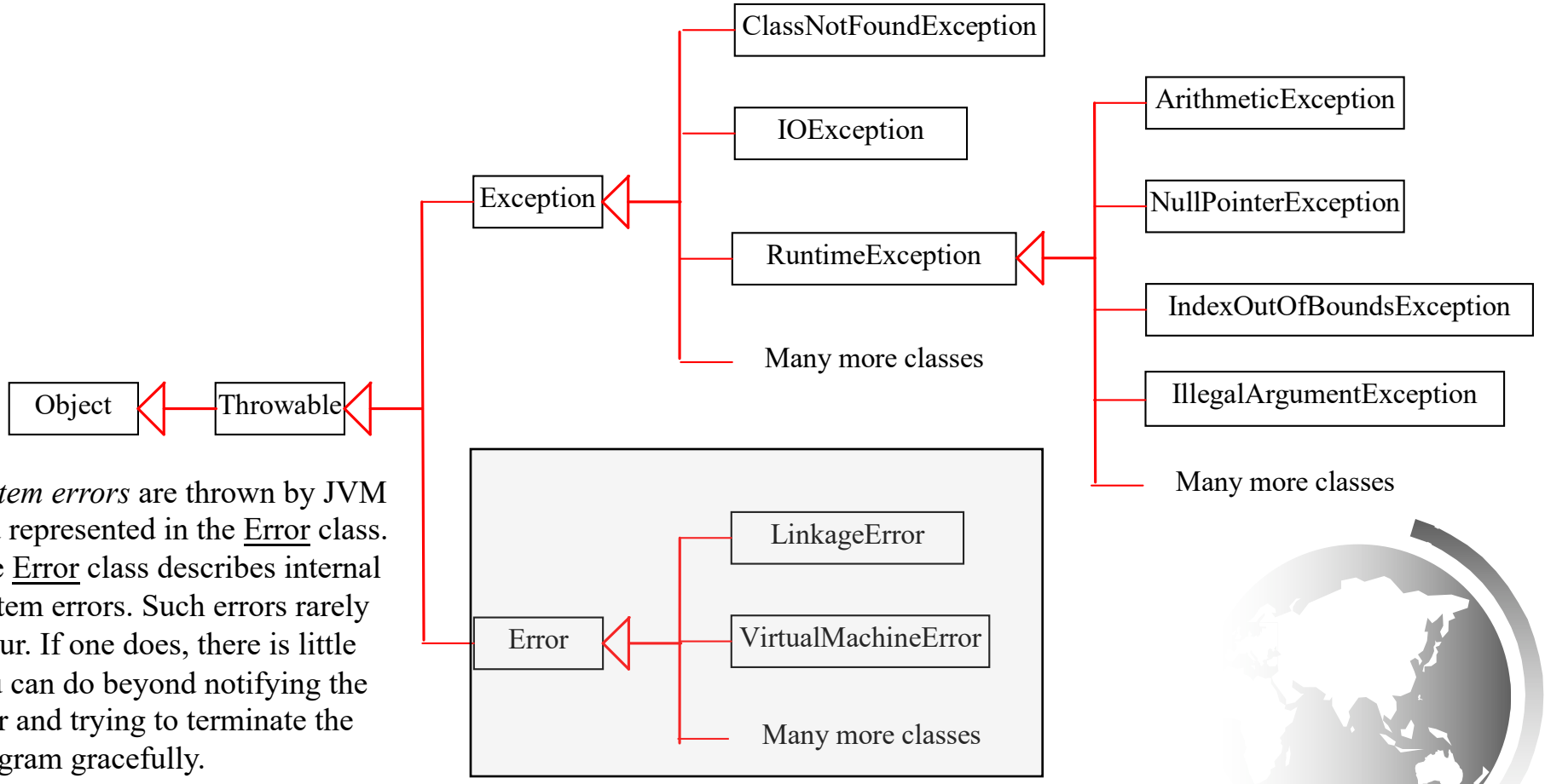
Exception Types

Exceptions are objects, and objects are defined using classes. The root class for exceptions is **java.lang.Throwable**.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

System Errors

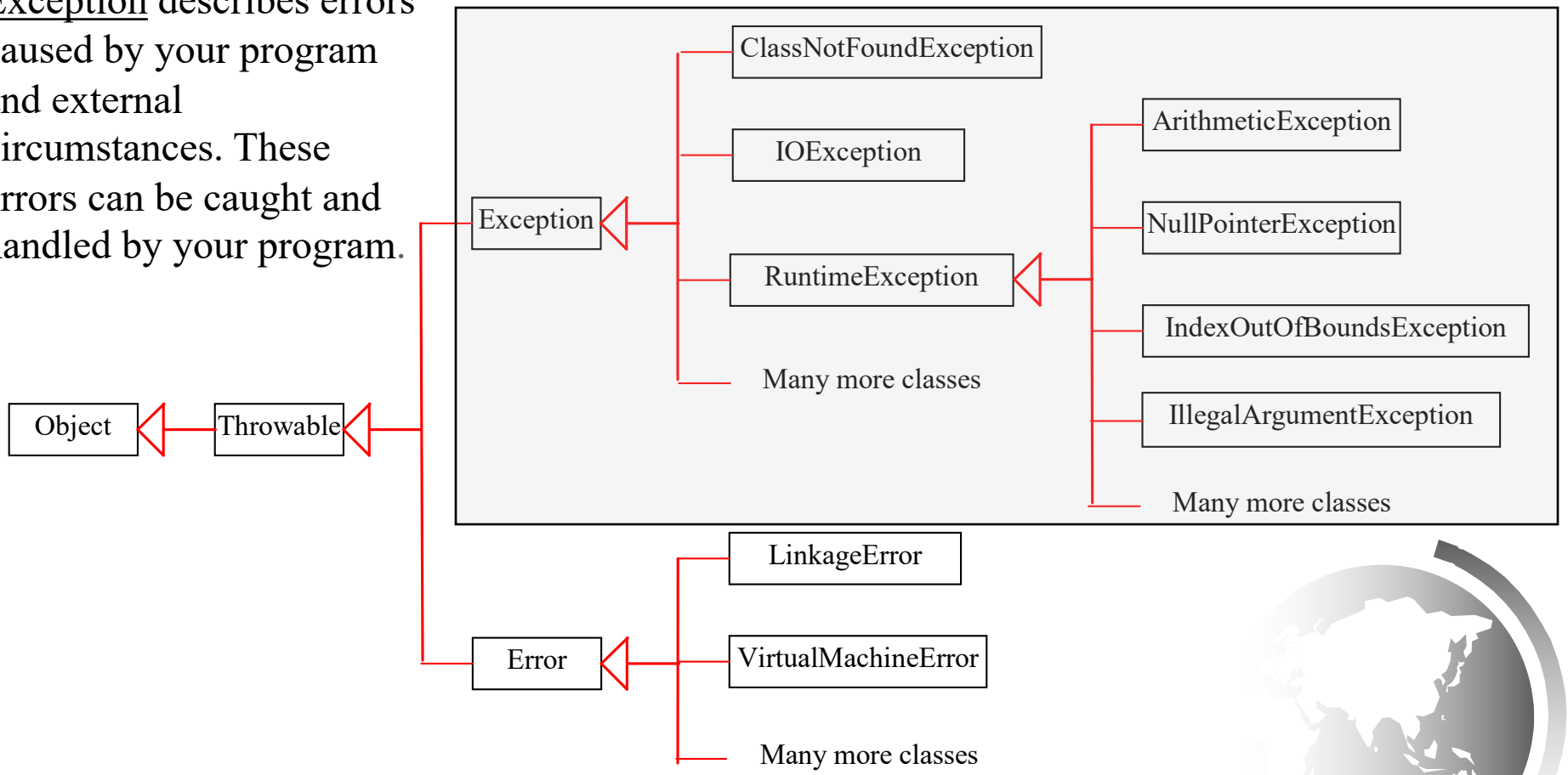


System errors are thrown by JVM and represented in the Error class. The Error class describes internal system errors. Such errors rarely occur. If one does, there is little you can do beyond notifying the user and trying to terminate the program gracefully.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

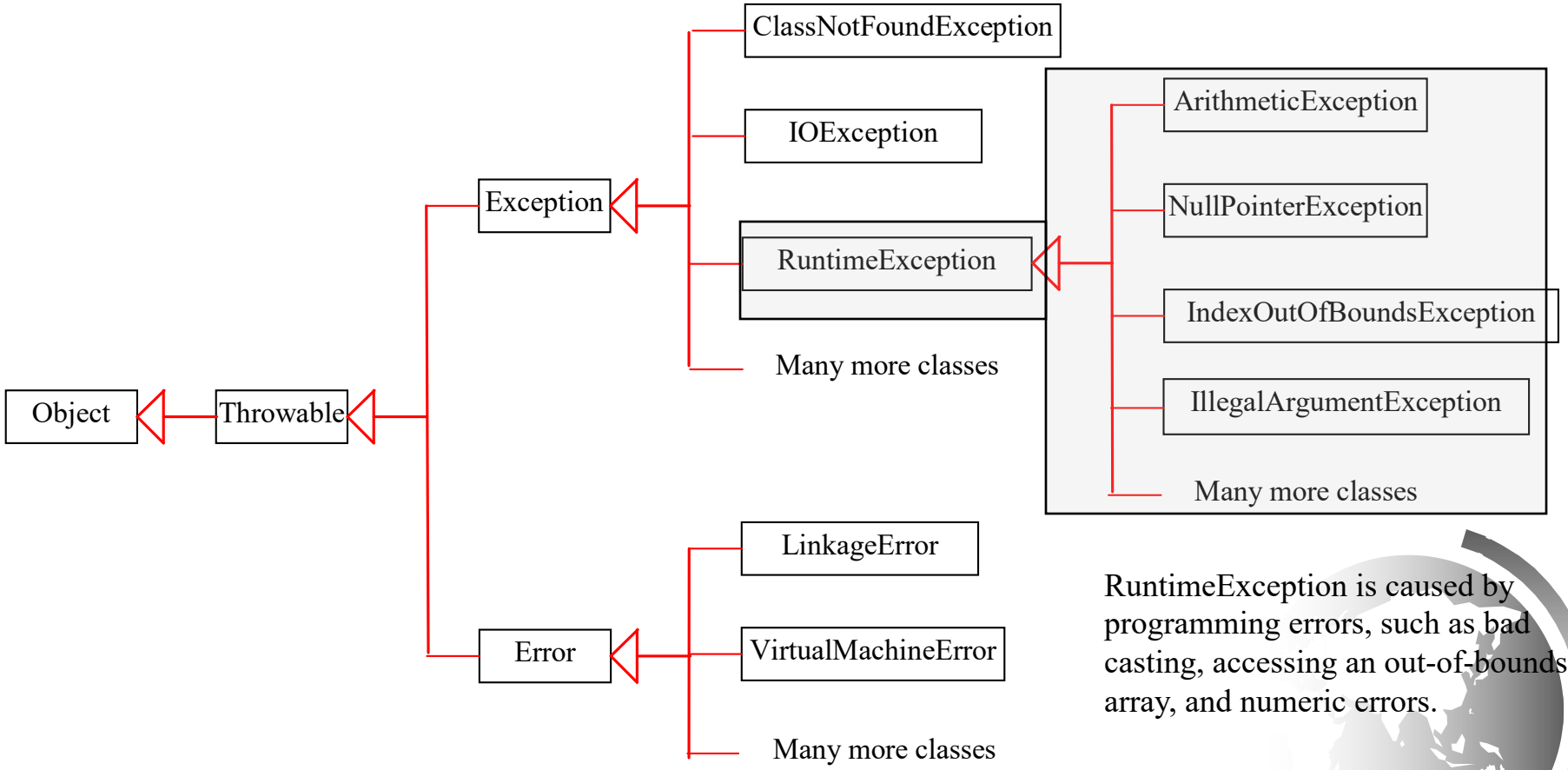
Exceptions

Exception describes errors caused by your program and external circumstances. These errors can be caught and handled by your program.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Runtime Exceptions



RuntimeException is caused by programming errors, such as bad casting, accessing an out-of-bounds array, and numeric errors.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Checked Exceptions vs. Unchecked Exceptions

RuntimeException, Error and their subclasses are known as *unchecked exceptions*. All other exceptions are known as *checked exceptions*, meaning that the compiler forces the programmer to check and deal with the exceptions.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

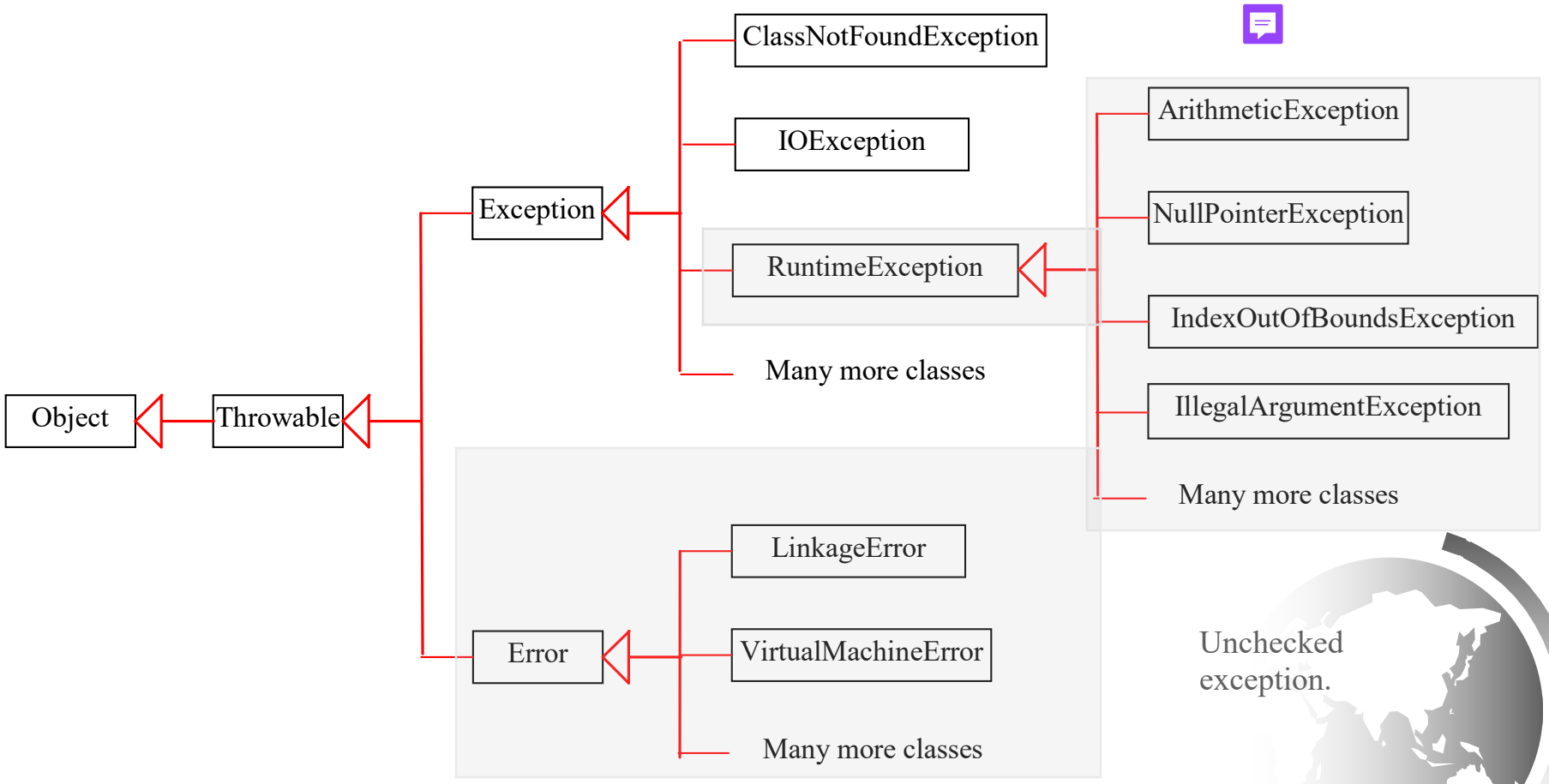
Unchecked Exceptions

In most cases, unchecked exceptions reflect programming logic errors that are not recoverable. For example, a NullPointerException is thrown if you access an object through a reference variable before an object is assigned to it; an IndexOutOfBoundsException is thrown if you access an element in an array outside the bounds of the array. These are the logic errors that should be corrected in the program. Unchecked exceptions can occur anywhere in the program. To avoid cumbersome overuse of try-catch blocks, Java does not mandate you to write code to catch unchecked exceptions.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

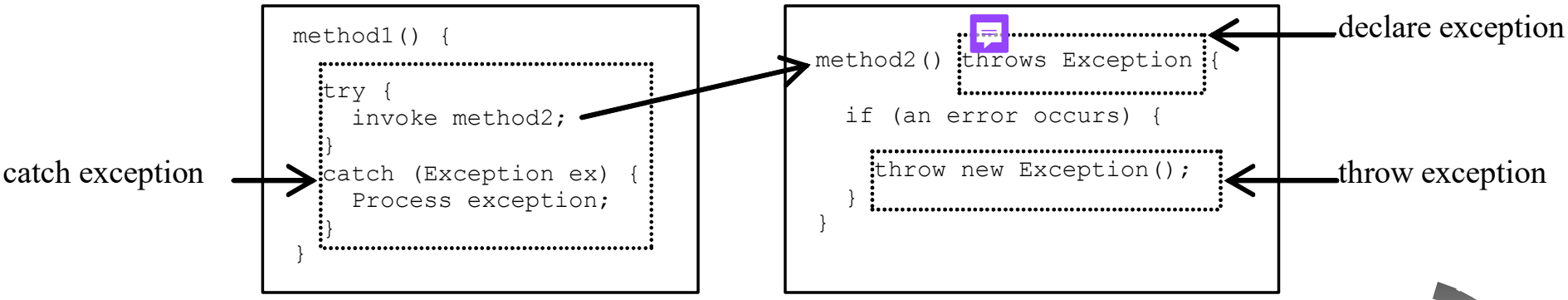
Unchecked Exceptions



Unchecked exception.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Declaring, Throwing, and Catching Exceptions



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Declaring Exceptions

Every method must state the types of checked exceptions it might throw. This is known as *declaring exceptions*.

```
public void myMethod()  throws IOException
```

```
public void myMethod()
   throws IOException, OtherException
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Throwing Exceptions

When the program detects an error, the program can create an instance of an appropriate exception type and throw it. This is known as *throwing an exception*. Here is an example,

```
throw new TheException();
```

```
TheException ex = new TheException();
throw ex;
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Throwing Exceptions Example

```
   /** Set a new radius */
public void setRadius(double newRadius)
    throws IllegalArgumentException {
  if (newRadius >= 0)
    radius =  newRadius;
  else
    throw new IllegalArgumentException(
      "Radius cannot be negative");
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

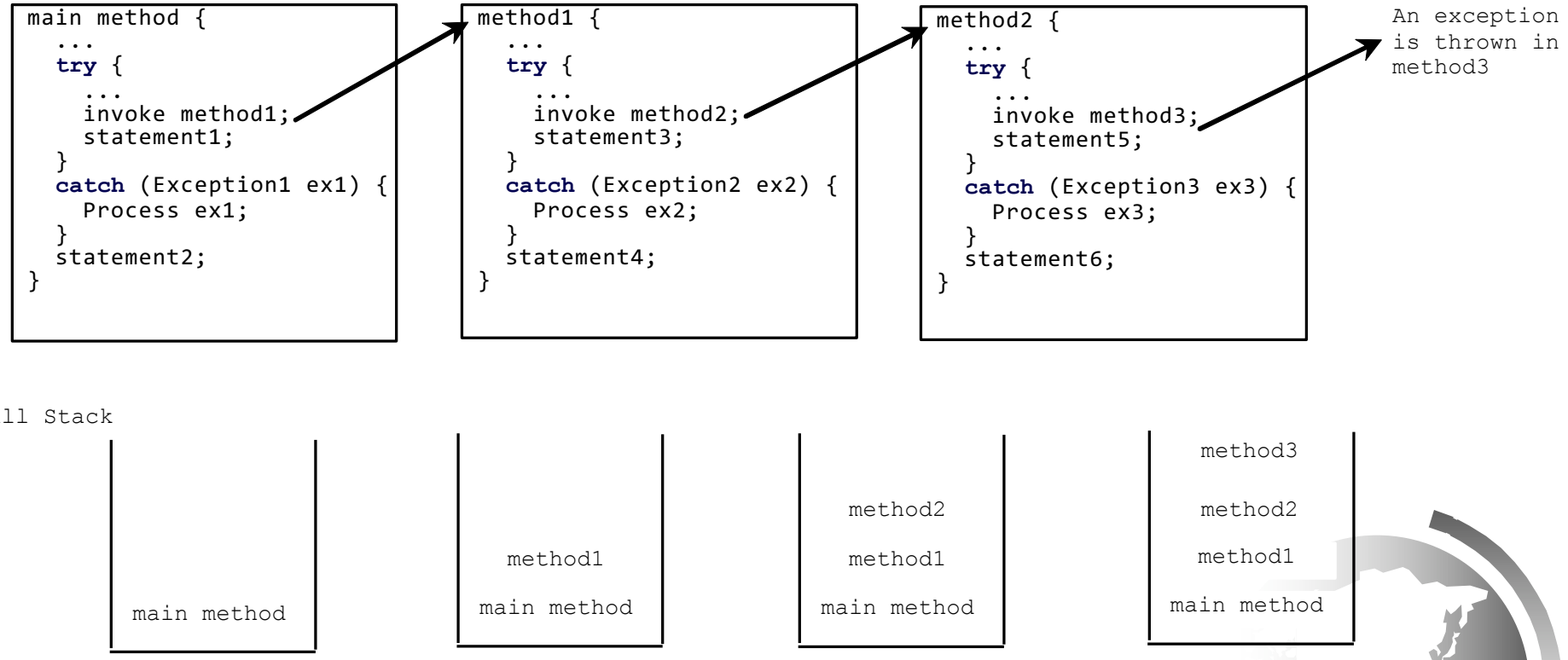
Catching Exceptions

```
try {
  statements;  // Statements that may throw exceptions
}
catch (Exception1 exVar1) {
  handler for exception1;
}
catch (Exception2 exVar2) {
  handler for exception2;
}
...
catch (ExceptionN exVar3) {
  handler for exceptionN;
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Catching Exceptions



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Catch or Declare Checked Exceptions

Suppose p2 is defined as follows:

```
void p2() throws IOException {
  if (a file does not exist) {
     throw new IOException("File does not exist");
  }

  ...
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Catch or Declare Checked Exceptions

Java forces you to deal with checked exceptions. If a method declares a checked exception (i.e., an exception other than Error or RuntimeException), you must invoke it in a try-catch block or declare to throw the exception in the calling method. For example, suppose that method p1 invokes method p2 and p2 may throw a checked exception (e.g., IOException), you have to write the code as shown in (a) or (b).

```
void p1() {
  try {
    p2();
  }
  catch (IOException ex) {
    ...
  }
}
```

(a)

```
void p1() throws IOException {

  p2();

}
```

(b)

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Order of Exception Handlers

- The order in which exceptions are specified in **catch** blocks is important.
- *A compile error* will result if a catch block for a superclass type appears before a catch block for a subclass type.
- For example, the ordering in (a) is erroneous, because **RuntimeException** is a subclass of **Exception**.
- The correct ordering should be as shown in (b).

```
try {
  ...
}
catch (Exception ex) {
  ...
}
catch (RuntimeException ex) {
  ...
}
```

(a) Wrong order

```
try {
  ...
}
catch (RuntimeException ex) {
  ...
}
catch (Exception ex) {
  ...
}
```

(b) Correct order



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Example: Declaring, Throwing, and Catching Exceptions

- Objective: This example demonstrates declaring, throwing, and catching exceptions by modifying the setRadius method in the Circle class defined in Chapter 9. The new setRadius method throws an exception if radius is negative.

CircleWithException

TestCircleWithException



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Exercise #1:

Suppose that **statement2** causes an exception in the following **try-catch** block:

```
try {
  statement1;
  statement2;
  statement3;
}
catch (Exception1 ex1) {
}
catch (Exception2 ex2) {
}

statement4;
```

- Will **statement3** be executed?

  No!

- If the exception is not caught, will **statement4** be executed?

  No!

- If the exception is caught in the **catch** block, will **statement4** be executed?

  Yes!



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Rethrowing Exceptions

```
try {
  statements;
}
catch(TheException ex) {
  perform operations before exits;
  throw ex;
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

The `finally` Clause

```
try {
  statements;
}
catch(TheException ex) {
  handling ex;
}
finally {
  finalStatements;
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

animation

Trace a Program Execution

Suppose no exceptions in the statements

```
try {
  statements;
}
catch(TheException ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

animation

Trace a Program Execution

```
try {
  statements;
}
catch(TheException ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

The final block is always executed



Copyright © 2022 Pearson Education Ltd. All rights reserved.

animation

Trace a Program Execution

```
try {
  statements;
}
catch(TheException ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

Next statement in the method is executed



Copyright © 2022 Pearson Education Ltd. All rights reserved.

animation

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

Suppose an exception of type Exception1 is thrown in statement2



Copyright © 2022 Pearson Education Ltd. All rights reserved.

animation

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

The exception is handled.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

animation

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

The final block is always executed.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

animation

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
finally {
  finalStatements;
}

Next statement;
```

The next statement in the method is now executed.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

animation

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}

Next statement;
```

statement2 throws an exception of type Exception2.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

animation

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}

Next statement;
```

Handling exception



Copyright © 2022 Pearson Education Ltd. All rights reserved.

animation

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}

Next statement;
```

Execute the final block



Copyright © 2022 Pearson Education Ltd. All rights reserved.

animation

Trace a Program Execution

```
try {
  statement1;
  statement2;
  statement3;
}
catch(Exception1 ex) {
  handling ex;
}
catch(Exception2 ex) {
  handling ex;
  throw ex;
}
finally {
  finalStatements;
}

Next statement;
```

Rethrow the exception and control is transferred to the caller



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Cautions When Using Exceptions

- Exception handling separates error-handling code from normal programming tasks, thus making programs easier to read and to modify.
- Be aware, however, that exception handling usually requires more time and resources because it requires instantiating a new exception object, rolling back the call stack, and propagating the errors to the calling methods.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

When to Throw Exceptions

- An exception occurs in a method.
- If you want the exception to be processed by its caller, you should create an exception object and throw it.
- If you can handle the exception in the method where it occurs, there is no need to throw it.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

When to Use Exceptions

When should you use the try-catch block in the code? You should use it to deal with unexpected error conditions. Do not use it to deal with simple, expected situations. For example, the following code

```
try {
  System.out.println(refVar.toString());
}
catch (NullPointerException ex) {
  System.out.println("refVar is null");
}
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

When to Use Exceptions

is better to be replaced by

```
if (refVar != null)
  System.out.println(refVar.toString());
else
  System.out.println("refVar is null");
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Defining Custom Exception Classes

- Use the exception classes in the API whenever possible.
- Define custom exception classes if the predefined classes are not sufficient.
- Define custom exception classes by extending Exception or a subclass of Exception.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Custom Exception Class Example

In Listing 13.8, the setRadius method throws an exception if the radius is negative. Suppose you wish to pass the radius to the handler, you have to create a custom exception class.

InvalidRadiusException

CircleWithRadiusException

TestCircleWithRadiusException



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Text I/O



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Objectives

- To discover file/directory properties, to delete and rename files/directories, and to create directories using the **File** class.
- To write data to a file using the **PrintWriter** class.
- To use try-with-resources to ensure that the resources are closed automatically.
- To read data from a file using the **Scanner** class.
- To understand how data is read using a **Scanner.**
- To develop a program that replaces text in a file.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

The File Class

- The File class is intended to provide an abstraction that deals with most of the machine-dependent complexities of files and path names in a machine-independent fashion.
- The filename is a string.
- The File class is a wrapper class for the file name and its directory path.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

The File Class

- The **File** class contains the methods
  – for obtaining file and directory properties
  – for renaming and deleting files and directories.
- However, *the **File** class does not contain the methods for reading and writing file content*



Copyright © 2022 Pearson Education Ltd. All rights reserved.

File Names

- *absolute file name* (or *full name*)
  - contains a file name with its complete path and drive letter.
  - Ex:

    c:\book\Welcome.java
- *relative file name*
  - contains a file name in relation to the current working directory
  - If the current working directory is **c:\book**, **Welcome.java** is a relative file name



Copyright © 2022 Pearson Education Ltd. All rights reserved.

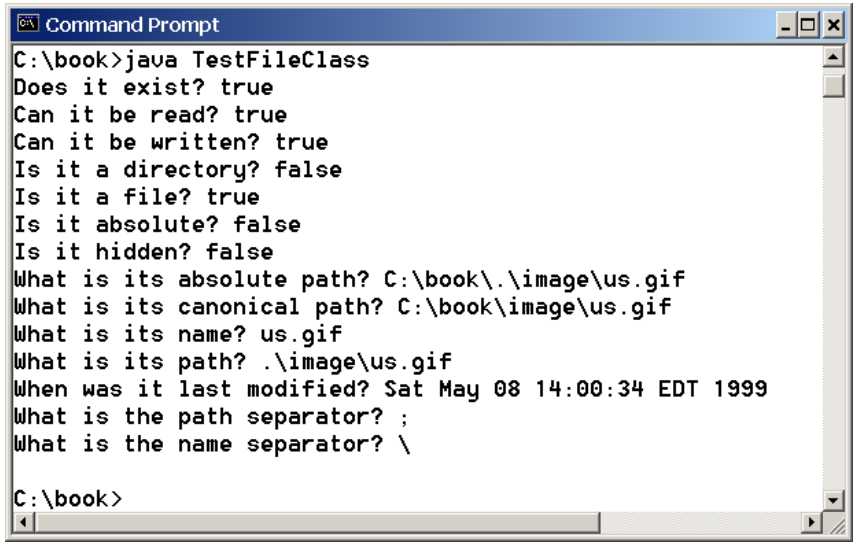
Obtaining file properties and manipulating file

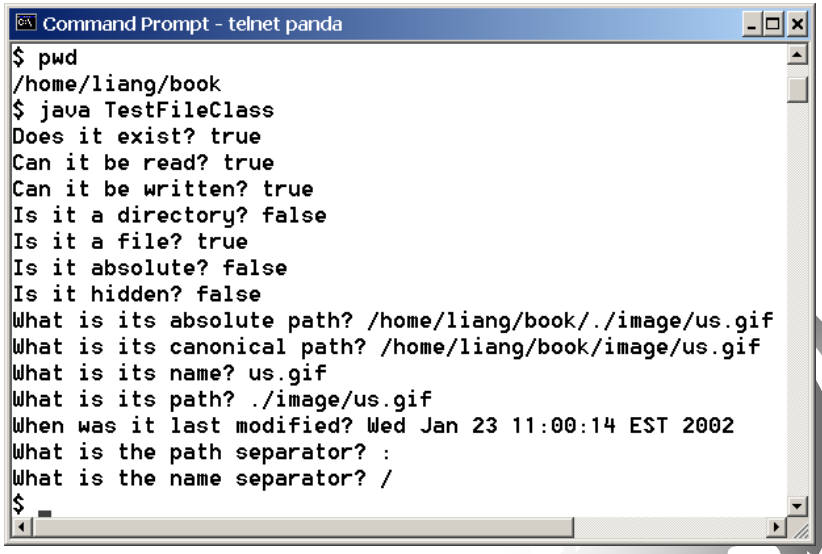
java.io.File	
`+File(pathname: String)`	Creates a `File` object for the specified path name. The path name may be a directory or a file.
`+File(parent: String, child: String)`	Creates a `File` object for the child under the directory parent. The child may be a file name or a subdirectory.
`+File(parent: File, child: String)`	Creates a `File` object for the child under the directory parent. The parent is a `File` object. In the preceding constructor, the parent is a string.
`+exists(): boolean`	Returns true if the file or the directory represented by the `File` object exists.
`+canRead(): boolean`	Returns true if the file represented by the `File` object exists and can be read.
`+canWrite(): boolean`	Returns true if the file represented by the `File` object exists and can be written.
`+isDirectory(): boolean`	Returns true if the `File` object represents a directory.
`+isFile(): boolean`	Returns true if the `File` object represents a file.
`+isAbsolute(): boolean`	Returns true if the `File` object is created using an absolute path name.
`+isHidden(): boolean`	Returns true if the file represented in the `File` object is hidden. The exact definition of *hidden* is system-dependent. On Windows, you can mark a file hidden in the File Properties dialog box. On Unix systems, a file is hidden if its name begins with a period(.) character.
`+getAbsolutePath(): String`	Returns the complete absolute file or directory name represented by the `File` object.
`+getCanonicalPath(): String`	Returns the same as `getAbsolutePath()` except that it removes redundant names, such as "." and "..", from the path name, resolves symbolic links (on Unix), and converts drive letters to standard uppercase (on Windows).
`+getName(): String`	Returns the last name of the complete directory and file name represented by the `File` object. For example, new `File("c:\\book\\test.dat").getName()` returns `test.dat`.
`+getPath(): String`	Returns the complete directory and file name represented by the `File` object. For example, new `File("c:\\book\\test.dat").getPath()` returns `c:\book\test.dat`.
`+getParent(): String`	Returns the complete parent directory of the current directory or the file represented by the `File` object. For example, new `File("c:\\book\\test.dat").getParent()` returns `c:\book`.
`+lastModified(): long`	Returns the time that the file was last modified.
`+length(): long`	Returns the size of the file, or 0 if it does not exist or if it is a directory.
`+listFile(): File[]`	Returns the files under the directory for a directory `File` object.
`+delete(): boolean`	Deletes the file or directory represented by this `File` object.The method returns true if the deletion succeeds.
`+renameTo(dest: File): boolean`	Renames the file or directory represented by this `File` object to the specified name represented in dest. The method returns true if the operation succeeds.
`+mkdir(): boolean`	Creates a directory represented in this `File` object. Returns true if the the directory is created successfully.
`+mkdirs(): boolean`	Same as `mkdir()` except that it creates directory along with its parent directories if the parent directories do not exist.

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Problem: Explore File Properties

Objective: Write a program that demonstrates how to create files in a platform-independent way and use the methods in the File class to obtain their properties. The following figures show a sample run of the program on Windows and on Unix.





TestFileClass

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Text I/O

A File object encapsulates the properties of a file or a path, but does not contain the methods for reading/writing data from/to a file. In order to perform I/O, you need to create objects using appropriate Java I/O classes. The objects contain the methods for reading/writing data from/to a file. This section introduces how to read/write strings and numeric values from/to a text file using the Scanner and PrintWriter classes.



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Writing Data Using PrintWriter

java.io.PrintWriter	
+PrintWriter(filename: String)	Creates a PrintWriter for the specified file.
+print(s: String): void	Writes a string.
+print(c: char): void	Writes a character.
+print(cArray: char[]): void	Writes an array of character.
+print(i: int): void	Writes an int value.
+print(l: long): void	Writes a long value.
+print(f: float): void	Writes a float value.
+print(d: double): void	Writes a double value.
+print(b: boolean): void	Writes a boolean value.
Also contains the overloaded println methods.	A println method acts like a print method; additionally it prints a line separator. The line separator string is defined by the system. It is \r\n on Windows and \n on Unix.
Also contains the overloaded printf methods.	The printf method was introduced in §4.6, "Formatting Console Output and Strings."

WriteData

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Try-with-resources

Programmers often forget to close the file. JDK 7 provides the followings new try-with-resources syntax that automatically closes the files.

```
try (declare and create resources) {
  Use the resource to process the file;
}
```

WriteDataWithAutoClose



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Reading Data Using Scanner

- To read from a file, create a **Scanner** for a file, as follows:

```
Scanner input = new Scanner(new File(filename))
```



Copyright © 2022 Pearson Education Ltd. All rights reserved.

Reading Data Using Scanner

java.util.Scanner	
+Scanner(source: File)	Creates a Scanner object to read data from the specified file.
+Scanner(source: String)	Creates a Scanner object to read data from the specified string.
+close()	Closes this scanner.
+hasNext(): boolean	Returns true if this scanner has another token in its input.
+next(): String	Returns next token as a string.
+nextByte(): byte	Returns next token as a byte.
+nextShort(): short	Returns next token as a short.
+nextInt(): int	Returns next token as an int.
+nextLong(): long	Returns next token as a long.
+nextFloat(): float	Returns next token as a float.
+nextDouble(): double	Returns next token as a double.
+useDelimiter(pattern: String): Scanner	Sets this scanner's delimiting pattern.

ReadData

Copyright © 2022 Pearson Education Ltd. All rights reserved.

Problem: Replacing Text

Write a class named ReplaceText that replaces a string in a text file with a new string. The filename and strings are passed as command-line arguments as follows:

```
java ReplaceText sourceFile targetFile oldString newString
```

For example, invoking

```
java ReplaceText FormatString.java t.txt StringBuilder StringBuffer
```

replaces all the occurrences of StringBuilder by StringBuffer in FormatString.java and saves the new file in t.txt.

ReplaceText



Copyright © 2022 Pearson Education Ltd. All rights reserved.